Technical Architecture and Implementation Specification for a High-Performance JSML Browser Engine

The engineering of a modern browser engine, even one intended as a simplified imitation of prevailing standards, constitutes an exhaustive exercise in systems programming, concurrent architecture, and hardware-accelerated graphics. This report provides a comprehensive blueprint for the implementation of a JSML-based engine, utilizing a C++ core integrated with the SDL3 graphics subsystem, the Lua 5.4 scripting runtime, and the libcurl networking stack. The project is designed around a strictly defined seven-day development cycle, necessitating a modular approach that prioritizes high-throughput layout algorithms and a robust "Snapshot and Commit" state management model.

Theoretical Foundation and the JSML Document Specification

The structural core of the engine is defined by JSML, a specialized language that conflates structural markup and stylistic declarations into a unified JSON-like hierarchy. Unlike the traditional separation of concerns found in the HTML/CSS paradigm, where a separate Cascading Style Sheets engine must resolve inheritance and specificity, JSML facilitates a direct mapping between the document tree and the rendering instructions.[2] This design significantly reduces the computational overhead associated with the CSS cascade, allowing the engine to focus its resources on geometric layout and GPU-bound drawing operations.

Resilient Lexical Analysis with nlohmann::json

The ingestion of raw JSML data is handled by the nlohmann::json library, which provides a high-performance, predictive LL(1) parser.[3] A critical implementation detail involves the handling of technical ambiguities within the JSML specification. For instance, while standard JSON (ECMA-404) forbids comments, JSML explicitly permits C-style comments for documentation and benchmarking directives.[1] Consequently, the parser must be configured with specific flags to treat comments and trailing commas as ignorable whitespace to ensure interoperability with user-authored files.[3]

The following table summarizes the identified inconsistencies in the JSML specification and the proposed engineering resolutions:

Ambiguity	Source	Engineering Resolution
Syntax Inconsistency	Page 1, Page 2	The parser must utilize a resilient object structure to handle properties placed both

		inside and outside flex arrays.
Tag Property Type	Page 1, Page 2	The engine adopts a variant-based ID system where the tag attribute can be either an integer or a string, ensuring getElemsByTag functions correctly.[1]
Spacing Definition	Page 2	Spacing is implemented as a sibling gap (equivalent to CSS gap) rather than internal padding or margins.
Resource Pathing	Page 1	A dedicated URI resolver is implemented to handle protocol-less paths (e.g., /script.lua) by resolving them against the base URL.
Image Load State	Page 2	This is implemented as an asynchronous onload event to trigger texture telemetry and UI updates.

Implementation of the Resilient Parser

The C++ backend utilizes a custom wrapper for the nlohmann::json::parse function. This wrapper ensures that the ignore_comments and ignore_trailing_commas parameters are set to true, providing a robust ingestion layer.[3]

C++

```
// Robust JSML Parsing Logic
nlohmann::json parse_jsml(const std::string& raw_input) {
  try {
    // Fourth argument is ignore_comments, fifth is ignore_trailing_commas
    return nlohmann::json::parse(raw_input, nullptr, true, true, true);
  } catch (const nlohmann::json::parse_error& e) {
    // Log the error and return a minimal valid JSML root
    std::cerr << "JSML Parse Error: " << e.what() << std::endl;
    return nlohmann::json::object();
  }
}
```

This parsing stage transforms the raw JSML into an internal Document Object Model (DOM),

where each node represents either a container (flexv, flexh) or a terminal content element (text, image).[1]

The Layout Engine: Algorithmic Geometry and Flexbox Principles

Once the logical DOM is established, the engine must perform a layout pass to calculate the geometric bounds of each element. This process transforms the logical tree into a "Layout Tree," where each node is assigned an absolute (x, y) coordinate and a definitive $(width, height)$. The implementation follows a simplified subset of the Flexbox algorithm, optimized for high-frequency benchmarking of millions of elements.[1]

The Two-Pass Recursive Layout Process

The layout engine executes in two distinct recursive passes:

1. **The Measurement Pass (Post-Order Traversal)**: This pass determines the intrinsic size of terminal nodes. For text elements, the engine queries the font renderer (utilizing FreeType via SDL3_ttf) to determine the bounding box of the string at a given fontsize.[1] For images, the size is derived from the metadata of the decoded bitmap textures.
2. **The Distribution Pass (Pre-Order Traversal)**: This pass allocates space within containers. For a flexv container, the primary axis is vertical.[2] The height of the container is calculated as the sum of the heights of its children plus the accumulated spacing between them.[1]

The mathematical representation for a vertical container's height is:

$$Height_{container} = \sum_{i=1}^{n} Height_{child_i} + (n-1) \times Spacing$$

If a container has fixed dimensions, the engine calculates the "positive free space" available after the initial layout.[1] This space is then redistributed among eligible children based on their flex-grow factors.[1] If the children exceed the available space, they may shrink according to their flex-shrink values or overflow the container.[1]

Handling Flex Growth Factors

The distribution of free space is a proportional operation. For each child i with a grow factor G_i, the additional size S_i is calculated as follows:

$$S_i = \left(\frac{G_i}{\sum_{j=1}^{n} G_j} \right) \times FreeSpace_{total}$$

This logic ensures that elements expand proportionally to fill the available container size, a key requirement for responsive UI design.[8] The implementation must account for constraints such as min-width and max-height during this phase.[5]

Hardware-Accelerated Rendering Pipeline with SDL3

GPU

To achieve the performance required for rendering millions of elements, the engine utilizes the SDL3 GPU API, bypassing the overhead associated with traditional CPU-bound drawing and the aging SDL_Renderer.[12] The SDL3 GPU API provides a modern abstraction for Vulkan, Metal, and D3D12, allowing for explicit control over memory allocation and command submission.[12]

Batching and Vertex Buffer Management

The primary bottleneck in high-performance 2D rendering is the overhead of draw calls.[14] The engine optimizes this by implementing **Render Batching**.[1] All elements that share a rendering state—such as rectangles of the same color or text using the same font atlas—are grouped into a single vertex buffer.[1]

For massive element counts, the engine utilizes **Instanced Rendering**.[1] A single rectangle mesh is stored in a GPU buffer, and the engine submits an array of "instance data" containing the unique position, size, and color for each element.[1] This allows the GPU to process thousands of elements in a single transaction.[1]

GPU Resource	SDL3 Function	Engineering Purpose
Graphics Device	SDL_CreateGPUDevice	Access to the underlying graphics driver (Vulkan/Metal).[12]
Shaders	SDL_CreateGPUShader	Execution of custom vertex and fragment logic for UI primitives.[12]
Storage Buffers	SDL_CreateGPUBuffer	Holding instance data for millions of layout rectangles.[1]
Textures	SDL_UploadToGPUTexture	Storing font atlases and decoded images for rapid sampling.[12]
Command Buffers	SDL_AcquireGPUCommandBuffer	Collecting draw instructions for batch submission to the GPU.[12]

Text Rendering via Texture Atlases

Text is rendered using a **Texture Atlas**—a single large image containing the rasterized glyphs of a font.[1] The engine uses SDL3_ttf to render glyphs into a surface, which is then uploaded to the GPU.[6] When drawing text strings, the engine generates a series of quads, where each quad's texture coordinates point to the specific region of the atlas corresponding to the character.[20] This approach allows text to be batched similarly to geometric primitives, drastically reducing the number of texture swaps per frame.[15]

The Embedded Scripting Runtime and Lua 5.4 Integration

The browser's interactivity is facilitated by an embedded Lua 5.4 runtime, integrated via the sol2 header-only library.[1] Lua is selected for its lightweight nature, exceptional performance in embedded contexts, and its simple, 21-keyword syntax.[2]

The Snapshot and Commit Model

The engine adopts a "Snapshot and Commit" model for DOM manipulation to ensure thread safety and architectural consistency.[1] In this paradigm, the scripting environment is isolated from the live rendering state.

1. **Snapshot**: When a Lua script requests an element via browser.getElem(id), the C++ engine generates a serialized snapshot of the element's properties as a native Lua table.[1]
2. **Modification**: The script modifies the Lua table locally. These changes are immutable relative to the UI until a commit occurs.[2]
3. **Commit**: The script calls browser.updateElem(id, table), which "commits" the state changes back to the engine, marking the affected Layout Tree nodes as "dirty" and scheduling a reflow.[1]

The Lua Runtime API Implementation

The C++ engine exports the following functions to the Lua state to enable document manipulation and telemetry [2]:

C++

```
// Binding the Browser Table with sol2
sol::table browser = lua.create_table("browser");

browser.set_function("getElemsByTag",(std::string tag) {
  // DFS traversal of the DOM to return a table of unique IDs
  return engine::get_ids_by_tag(tag);
});

browser.set_function("getElem",(int id) {
  // Serializes the C++ Element object to a sol::table snapshot
  return engine::get_element_snapshot(id);
});

browser.set_function("updateElem",(int id, sol::table table) {
  // Deserializes the table back to the C++ object and marks layout dirty
```

```
    engine::commit_element_update(id, table);
});

browser.set_function("fetch",(std::string url, sol::function callback) {
    // Initiates an asynchronous libcurl request with a Lua callback
    engine::async_fetch(url, callback);
});
```

Detailed Lua Scripting Logic

The following script illustrates the expected Lua code for the runtime, demonstrating how a script interacts with the Snapshot-Commit model to update UI components based on network responses [2]:

Lua

```
-- Example Lua Script for Interactive Logic
local elementIds = browser.getElemsByTag("tag-1")
local buttonId = elementIds

-- Register an asynchronous click handler
browser.addClickHandler(buttonId, function()
    browser.log("Button clicked, fetching data...")

    -- Perform an asynchronous fetch
    browser.fetch("/api/data", function(response)
        -- Retrieve the current state snapshot
        local element = browser.getElem(buttonId)

        -- Modify the snapshot locally
        element["text"]["content"] = "Response: ".. response["data"]
        element["bgcolor"] = "#00FF00"

        -- Commit the changes back to the engine
        browser.updateElem(buttonId, element)

        local endTime = browser.getTime()
        browser.log("Update completed at: ".. endTime)
    end)
end)
```

Non-Blocking Networking and Asynchronous IO

The engine must support loading JSML pages, scripts, and images over the network without stalling the UI or layout threads. This is achieved by implementing a dedicated IO thread utilizing the libcurl multi-interface.[24]

The libcurl Multi-Interface Architecture

The libcurl multi-interface enables multiple simultaneous, non-blocking transfers within a single thread.[24] The networking workflow is structured as follows:

1. **Request Initiation**: When a script calls fetch, the engine creates a "curl easy handle," sets the necessary options (e.g., CURLOPT_URL, CURLOPT_WRITEFUNCTION), and adds it to a "curl multi handle".[24]
2. **The Multi-Loop**: The IO thread continuously calls curl_multi_perform and curl_multi_poll to process network traffic.[25]
3. **Completion and Callback**: When a transfer completes, the IO thread extracts the data, parses the JSON response using nlohmann::json, and posts a completion event to the Logic thread's task queue to execute the Lua callback.[1]

URI Resolution and Security Sandboxing

The engine must resolve relative paths relative to the base URL of the currently loaded document. A request for /image.png on a page loaded from http://example.com/site must resolve to http://example.com/image.png. Furthermore, for local file access, the engine implements a basic sandbox to ensure that JSML pages can only access files within the designated resources directory or the path of the original JSML file.

Concurrency, Threading, and Triple Buffering

To maintain a responsive interface during heavy benchmarking, the engine employs a triple-threaded execution model.[1]

Thread Distribution

Thread	Responsibility	Synchronization Mechanism
Main (UI) Thread	Windowing, input polling (SDL3), GPU draw calls.[12]	Consumes the "Front Buffer" from the Triple Buffer.[1]
Logic & Layout Thread	Lua VM execution, Flexbox calculations, DOM updates.[1]	Produces the "Back Buffer" for the Triple Buffer.[1]
IO & Networking Thread	Non-blocking libcurl polling, image decoding.[1]	Communicates via an asynchronous task queue.

Lock-Free Triple Buffering Implementation

Passing the "Render Tree"—a flattened list of draw commands—from the Logic thread to the Main thread requires a synchronization mechanism that avoids long-duration locks.[1] The engine utilizes a lock-free triple buffer, where three buffers are maintained: **Front**, **Spare**, and **Back**.[28] The synchronization is managed via atomic pointer swaps:

C++

```
// Atomic Triple Buffer Rotation
void publish_new_layout() {
   // Logic thread completes the Back Buffer
   // Atomic exchange with the Spare Buffer
   auto old_spare = buffer_state.exchange(back_buffer_ptr, std::memory_order_acq_rel);
   back_buffer_ptr = old_spare;
}

void consume_new_layout() {
   // UI thread checks if Spare has an update
   if (update_available.load(std::memory_order_acquire)) {
      auto old_front = front_buffer_ptr;
      front_buffer_ptr = buffer_state.exchange(old_front, std::memory_order_acq_rel);
   }
}
```

This ensures that the UI thread always renders from a stable, complete snapshot of the layout, while the Logic thread can immediately begin calculating the next frame without waiting for the GPU to finish its current pass.[29]

Strategic Implementation Roadmap

Building a browser engine in a week is a significant challenge that requires a disciplined, phase-based approach.[1]

Day-by-Day Technical Milestones

- **Day 1: Document Model and Lexical Layer**: Creation of the C++ DOM and the resilient JSML parser using nlohmann::json. Implementation of variant-based tag IDs and comment-ignoring logic.[1]
- **Day 2: Flexbox Layout Engine**: Implementation of the vertical and horizontal distribution math. Development of the text measurement stubs and recursive parent-child sizing.[1]
- **Day 3: SDL3 GPU Pipeline**: Initialization of the graphics device and swapchain. Implementation of the vertex/storage buffer batching and the rectangle shaders.[12]
- **Day 4: Embedded Scripting Integration**: Integration of Lua 5.4 and sol2. Implementation of the browser.getElem and browser.updateElem Snapshot-Commit

functions.[1]

- **Day 5: Networking and Async IO**: Integration of libcurl multi-interface. Implementation of the image loading pipeline and the URI resolution module.[1]
- **Day 6: Threading and Synchronization**: Moving the Lua runtime and layout to the Logic thread. Implementation of the lock-free triple-buffering system.[1]
- **Day 7: UI Implementation and Benchmarking**: Development of the tab bar and URL input using the engine's internal primitives. Fine-tuning of the batching logic for the "millions of elements" stress test.[1]

Systems Integration and Optimization

To meet the high-performance requirements, the engine must be compiled and linked with aggressive optimization strategies.

Build System and Toolchain

The project utilizes CMake for modularization, allowing the renderer, scripting engine, and networking stack to be developed as independent libraries. For the final binary, Link Time Optimization (LTO) is enabled to allow cross-module inlining of critical layout and drawing functions.[1]

Optimization Strategy	Engineering Impact
LTO (Link Time Optimization)	Potentially inlines recursive layout functions across source files, reducing function call overhead in tight loops.
Custom Allocators	Using mimalloc reduces fragmentation caused by the frequent allocation of small Lua tables and DOM nodes.[1]
Vectorization	SIMD instructions can be leveraged for bulk coordinate transformations during the layout distribution pass.
Batching	Consolidating thousands of rectangles into single draw calls via storage buffers minimizes GPU state churn.[12]

Benchmarking and Performance Profiling

The primary metric for success is the engine's ability to maintain high frame rates under a "millions of elements" load. This is achieved by ensuring that the complexity of the rendering pass is $O(DrawCalls)$ where $DrawCalls \ll Elements$ due to batching.[14] The layout pass is optimized via incremental reflow and "dirty marking," ensuring that only modified sub-trees are re-calculated.

Conclusions and Technical Summary

The proposed architecture for the toy browser engine represents a harmonious balance between structural correctness and execution speed. By strictly adhering to the "Snapshot and Commit" model, the engine ensures a stable environment for Lua scripting while maintaining the integrity of the rendering pipeline. The transition to a multi-threaded architecture with triple-buffered rendering is the most critical step for meeting high-performance benchmarks, as it effectively removes the scripting runtime from the critical path of the frame loop.[1] Success in this project is defined by the engine's ability to handle the "millions of elements" stress test, a feat achievable only through aggressive batching, clever concurrency, and the judicious use of modern GPU hardware acceleration via SDL3. The combination of modern C++ standards, robust libraries like sol2 and nlohmann::json, and a resilient architectural design provides the foundation necessary to deliver a professional-grade browser engine within an ambitious timeframe.[1]

Works cited

1. C++ Toy Browser Project Roadmap
2. LuaBrowser Runtime.pdf
3. parse - JSON for Modern C++ - Niels Lohmann, accessed April 1, 2026, https://json.nlohmann.me/api/basic_json/parse/
4. Comments - JSON for Modern C++, accessed April 1, 2026, https://json.nlohmann.me/features/comments/
5. Flexbox and grid layout - Raw Bits to React - Mintlify, accessed April 1, 2026, https://mintlify.com/renderffx/raw-bits-to-react/browser/box-model-flexbox
6. Rendering Text with Fonts (SDL3_ttf) - Free Pascal meets SDL, accessed April 1, 2026, https://freepascal-meets-sdl.net/rendering-text-with-fonts-sdl3_ttf-in-sdl3/
7. Basic concepts of flexbox - CSS - MDN Web Docs, accessed April 1, 2026, https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Flexible_box_layout/Basic_concepts
8. flex-grow - CSS - MDN Web Docs - Mozilla, accessed April 1, 2026, https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/flex-grow
9. Flexbox – Webflow Help Center, accessed April 1, 2026, https://help.webflow.com/hc/en-us/articles/33961260795155-Flexbox
10. flex-grow - CSS-Tricks, accessed April 1, 2026, https://css-tricks.com/almanac/properties/f/flex-grow/
11. Flexbox algorithm - QuirksMode, accessed April 1, 2026, http://quirksmode.org/css/flexbox-algorithm.html
12. SDL3/CategoryGPU - SDL Wiki, accessed April 1, 2026, https://wiki.libsdl.org/SDL3/CategoryGPU
13. sdl3_sys::gpu - Rust - Docs.rs, accessed April 1, 2026, https://docs.rs/sdl3-sys/latest/sdl3_sys/gpu/index.html
14. SDL GPU API Concepts: Sprite Batcher - Moonside Games, accessed April 1, 2026, https://moonside.games/posts/sdl-gpu-sprite-batcher/

15. Pop the geometry queue?/Optimize render with textures - SDL Development, accessed April 1, 2026, https://discourse.libsdl.org/t/pop-the-geometry-queue-optimize-render-with-textures/55067
16. Good practices for Batch rendering dynamic quads. : r/opengl - Reddit, accessed April 1, 2026, https://www.reddit.com/r/opengl/comments/td6hqo/good_practices_for_batch_rendering_dynamic_quads/
17. Need input on how to send and update the model matrix every frame on SDL3 GPU API, accessed April 1, 2026, https://www.reddit.com/r/sdl/comments/1mctbhr/need_input_on_how_to_send_and_update_the_model/
18. Sleen/FlexLayout: an implementation of Flexbox(Flexible Box) layout algorithm - GitHub, accessed April 1, 2026, https://github.com/Sleen/FlexLayout
19. SDL_gpu: it begins with a triangle. | by Hamdy Elzanqali - Medium, accessed April 1, 2026, https://hamdy-elzanqali.medium.com/let-there-be-triangles-sdl-gpu-edition-bd82cf2ef615
20. Rendering Text on the screen in real time - SDL Development, accessed April 1, 2026, https://discourse.libsdl.org/t/rendering-text-on-the-screen-in-real-time/20347
21. Renderer vs SDL_gpu for text rendering? - SDL Development - Simple Directmedia Layer, accessed April 1, 2026, https://discourse.libsdl.org/t/renderer-vs-sdl-gpu-for-text-rendering/31977
22. Sol2: Lua <-> C++ Binding Framework : r/cpp - Reddit, accessed April 1, 2026, https://www.reddit.com/r/cpp/comments/4a8gy7/sol2_lua_c_binding_framework/
23. Using Lua with C++ (and C) - Elias Daler, accessed April 1, 2026, https://edw.is/using-lua-with-cpp/
24. libcurl - API overview, accessed April 1, 2026, https://curl.se/libcurl/c/libcurl.html
25. How to do curl_multi_perform() asynchronously in C++? - Stack Overflow, accessed April 1, 2026, https://stackoverflow.com/questions/24288513/how-to-do-curl-multi-perform-asynchronously-in-c
26. libcurl - multi interface overview, accessed April 1, 2026, https://curl.se/libcurl/c/libcurl-multi.html
27. libcurl multi interface example - GitHub Gist, accessed April 1, 2026, https://gist.github.com/clemensg/4960504
28. Triple Buffering as a Concurrency Mechanism - GitHub, accessed April 1, 2026, https://github.com/remis-thoughts/blog/blob/master/triple-buffering/src/main/md/triple-buffering.md
29. Triple Buffer: Lock-free Concurrency Primitive | by Ng Song Guan ..., accessed April 1, 2026, https://medium.com/@sgn00/triple-buffer-lock-free-concurrency-primitive-611848627a1e
30. How I Learned to Stop Worrying and Love Juggling C++ Atomics | brilliantsugar,

accessed April 1, 2026, https://brilliantsugar.github.io/posts/how-i-learned-to-stop-worrying-and-love-juggling-c++-atomics/
31. Error with sol for Lua: items() is not a recognized container · Issue #4012 · nlohmann/json, accessed April 1, 2026, https://github.com/nlohmann/json/issues/4012
32. New, fastest JSON library for C++20 : r/cpp - Reddit, accessed April 1, 2026, https://www.reddit.com/r/cpp/comments/y37pd7/new_fastest_json_library_for_c20/
33. Confused about necessity of batch rendering - SDL Development, accessed April 1, 2026, https://discourse.libsdl.org/t/confused-about-necessity-of-batch-rendering/24048